

18. Spring Boot Actuator

18.1. Giới thiệu về Spring Boot Actuator

Spring Boot Actuator là một tính năng mạnh mẽ của Spring Boot, cung cấp các điểm cuối (endpoints) sẵn sàng cho môi trường sản xuất (production-ready) để giám sát và quản lý ứng dụng của bạn. Nó cho phép bạn dễ dàng truy cập các thông tin quan trọng về ứng dụng đang chạy, như trạng thái sức khỏe (health), metrics, thông tin môi trường, cấu hình, và nhiều hơn nữa, mà không cần phải viết bất kỳ mã nguồn tùy chỉnh nào.

Actuator đặc biệt hữu ích trong môi trường sản xuất, nơi bạn cần theo dõi hiệu suất, phát hiện vấn đề và thực hiện các hành động quản lý mà không cần khởi động lại ứng dụng.

18.2. Thêm Spring Boot Actuator vào dự án

Để sử dụng Spring Boot Actuator, bạn chỉ cần thêm dependency `spring-boot-starter-actuator` vào file `pom.xml` :

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Sau khi thêm dependency này, Spring Boot sẽ tự động cấu hình và expose một số endpoint Actuator mặc định.

18.3. Các Endpoint Actuator phổ biến

Actuator cung cấp nhiều endpoint khác nhau, mỗi endpoint cung cấp một loại thông tin cụ thể. Dưới đây là một số endpoint phổ biến:

Endpoint ID	Mô tả

<code>/actuator/health</code>	Cung cấp thông tin về trạng thái sức khỏe của ứng dụng (UP/DOWN), bao gồm trạng thái của database, message brokers, disk space, v.v.
<code>/actuator/info</code>	Hiển thị thông tin tùy chỉnh về ứng dụng (ví dụ: phiên bản, mô tả) được cấu hình trong <code>application.properties</code> .
<code>/actuator/metrics</code>	Cung cấp các metrics về hiệu suất ứng dụng (ví dụ: sử dụng CPU, bộ nhớ, số lượng request HTTP, thời gian phản hồi).
<code>/actuator/env</code>	Hiển thị tất cả các biến môi trường và thuộc tính cấu hình của Spring.
<code>/actuator/beans</code>	Liệt kê tất cả các Spring beans trong <code>ApplicationContext</code> .
<code>/actuator/mappings</code>	Hiển thị tất cả các ánh xạ request (request mappings) của ứng dụng.
<code>/actuator/loggers</code>	Cho phép xem và thay đổi cấp độ log của các logger trong thời gian chạy.
<code>/actuator/threaddump</code>	Thực hiện thread dump, hiển thị trạng thái của tất cả các thread đang chạy.
<code>/actuator/heapdump</code>	Tạo một heap dump của JVM, hữu ích cho việc phân tích bộ nhớ.
<code>/actuator/shutdown</code>	Cho phép tắt ứng dụng một cách graceful (mặc định bị vô hiệu hóa).

18.4. Cấu hình và Expose Endpoints

Theo mặc định, Spring Boot 2.x trở lên chỉ expose các endpoint `/health` và `/info` qua web. Để expose các endpoint khác, bạn cần cấu hình trong `application.properties` hoặc `application.yml`.

18.4.1. Expose qua Web (HTTP)

YAML

```

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,env,beans,loggers # Expose các endpoint
này
        # include: '*' # Expose tất cả các endpoint (không khuyến khích trong
production)
        # exclude: shutdown # Loại trừ endpoint shutdown
      base-path: /actuator # Thay đổi base path của Actuator (mặc định là
/actuator)
    endpoint:
      shutdown:
        enabled-by-default: false # Mặc định là false, cần bật rõ ràng nếu muốn
sử dụng
      health:
        show-details: always # Hiển thị chi tiết health (never, when-
authorized, always)
      info:
        enabled: true # Bật endpoint info

```

18.4.2. Expose qua JMX

Theo mặc định, tất cả các endpoint đều được expose qua JMX. Bạn có thể cấu hình tương tự:

YAML

```

management:
  endpoints:
    jmx:
      exposure:
        include: '*' # Expose tất cả qua JMX

```

18.4.3. Tùy chỉnh thông tin info endpoint

Bạn có thể thêm thông tin tùy chỉnh vào endpoint `/actuator/info` trong `application.properties` :

Plain Text

```

info.app.name=My Spring Boot Application
info.app.version=1.0.0

```

```
info.app.description=A sample Spring Boot application for demonstration.  
info.contact.email=contact@example.com
```

Hoặc sử dụng `build-info` từ Maven/Gradle plugin:

XML

```
<!-- Trong pom.xml -->  
<build>  
  <plugins>  
    <plugin>  
      <groupId>org.springframework.boot</groupId>  
      <artifactId>spring-boot-maven-plugin</artifactId>  
      <executions>  
        <execution>  
          <goals>  
            <goal>build-info</goal>  
          </goals>  
        </execution>  
      </executions>  
    </plugin>  
  </plugins>  
</build>
```

Sau đó, bạn có thể truy cập `/actuator/info` để xem thông tin build.

18.4.4. Tùy chỉnh Health Indicators

Spring Boot Actuator tự động cấu hình các Health Indicators cho các thành phần phổ biến như DataSource, Redis, RabbitMQ, v.v. Bạn cũng có thể tạo Health Indicator tùy chỉnh.

Ví dụ: Custom Health Indicator

Java

```
import org.springframework.boot.actuate.health.Health;  
import org.springframework.boot.actuate.health.HealthIndicator;  
import org.springframework.stereotype.Component;  
  
@Component  
public class CustomServiceHealthIndicator implements HealthIndicator {  
  
    private final MyCustomService myCustomService;
```

```

public CustomServiceHealthIndicator(MyCustomService myCustomService) {
    this.myCustomService = myCustomService;
}

@Override
public Health health() {
    if (myCustomService.isServiceUp()) {
        return Health.up().withDetail("serviceStatus",
"Operational").build();
    } else {
        return Health.down().withDetail("serviceStatus",
"Degraded").withDetail("error", "Service is not responding").build();
    }
}
}

```

18.5. Bảo mật Actuator Endpoints

Các endpoint Actuator có thể chứa thông tin nhạy cảm về ứng dụng và môi trường. Do đó, việc bảo mật chúng là cực kỳ quan trọng. Spring Boot Actuator tích hợp liền mạch với Spring Security.

Khi `spring-boot-starter-security` có trong classpath, tất cả các endpoint Actuator (trừ `/health` và `/info` nếu không có chi tiết nhạy cảm) sẽ được bảo vệ mặc định. Bạn cần xác thực để truy cập chúng.

Bạn có thể cấu hình bảo mật chi tiết hơn trong `SecurityConfig` của mình:

Java

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.boot.actuate.autoconfigure.security.servlet.EndpointRequest;

@Configuration
public class ActuatorSecurityConfig {

    @Bean

```

```

    public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception {
    http
        .authorizeHttpRequests(authz -> authz
            .requestMatchers(EndpointRequest.to("health",
"info")).permitAll() // Cho phép truy cập health, info

            .requestMatchers(EndpointRequest.toAnyEndpoint()).hasRole("ADMIN") // Các
            endpoint khác yêu cầu vai trò ADMIN
            .anyRequest().authenticated()
        )
        .httpBasic(); // Hoặc formLogin, OAuth2, v.v.
    return http.build();
}
}

```

18.6. Bản chất của Spring Boot Actuator

Bản chất của Spring Boot Actuator là cung cấp một bộ công cụ giám sát và quản lý ứng dụng toàn diện, sẵn sàng cho môi trường sản xuất, mà không yêu cầu cấu hình phức tạp. Nó hoạt động bằng cách tự động expose các endpoint HTTP hoặc JMX, cho phép các nhà vận hành và nhà phát triển dễ dàng thu thập thông tin về trạng thái, hiệu suất và hành vi của ứng dụng đang chạy. Actuator giúp giảm thiểu thời gian và công sức cần thiết để tích hợp các công cụ giám sát bên ngoài, đồng thời cung cấp khả năng tùy chỉnh và mở rộng để phù hợp với các yêu cầu cụ thể của từng ứng dụng. Điều này làm cho việc duy trì và gỡ lỗi các ứng dụng Spring Boot trong môi trường sản xuất trở nên hiệu quả hơn rất nhiều.